Exact Algebraic Characterization of Global Livelocks in Unidirectional Rings

Aly Farahat

Research & Development Division, Schweitzer Engineering Laboratories, 2350 NE Hopkins Court, Pullman, WA - 99163, U.S.A.

Abstract. This paper presents necessary and sufficient conditions for the livelock-freedom of a subclass of unidirectional rings in which processes are self-disabling; i.e., in each execution turn, the execution of an action of some process disables all actions of that process. We present our necessary and sufficient conditions in an algebraic setting that significantly simplifies reasoning about global livelocks in the local state space of the processes. The proposed approach enables a design and verification method that solely relies on local reasoning, thereby eliminating the need for reasoning about livelocks in the global state space. The proposed method has several applications including the design of global convergence for self-stabilizing systems and the verification of livelock-freedom in unidirectional rings. We evaluate the necessary and sufficient conditions in the context of several examples including Dijkstra's self-stabilizing token ring protocol whose proof is simplified.

1 Introduction

Livelocks are among the least understood types of concurrency flaws due to their dynamic, global and arbitrary nature [1]. We understand by a livelock/non-progress cycle, a sequence of undesirable global configurations that indefinitely repeat, thereby preventing further progress towards a global desirable configuration. Examples include process starvation for a shared resource [2], multi-token circulation in a mutual exclusion protocol [3], and the design of convergence in distributed protocols [4, 5]; i.e., protocols that recover to a desired behavior regardless of their initial configuration. To design livelock-free protocols irrespective of their number of processes, we investigate properties of global livelocks as perceived by a single process; i.e., local properties of livelocks. In this paper, we establish necessary and sufficient local conditions for global livelock-freedom in arbitrary-sized/parameterized unidirectional rings.

The problem of livelock-freedom in a parameterized network is generally undecidable. Apt and Kozen [6] reduce the non-halting problem to livelock-freedom verification of a parameterized network. Suzuki [7] establishes a similar result for parameterized unidirectional rings of symmetric processes. Consequently, most of livelock verification techniques are either (1) incomplete [8–12]; i.e., provide strictly sufficient conditions for livelock-freedom, (2) are applicable to a strict subclass of models of computation [13–16], (3) or are not amenable to automation by using manually designed ranking functions [8, 17, 18]. Due to their inherent incompleteness, the approaches in the first category do not exactly characterize livelocks, thereby preventing impossibility proofs similar to the results exhibited by Shukla et al. [19] for symmetric coloring protocols. The second category includes techniques that impose strong restrictions on unidirectional rings, thereby could not explore the convergence of simple protocols. For instance, all our case studies do not fall in the subclass captured by approaches in (2). Unlike the previous two techniques, manual methods in category (3) require human ingenuity to create strictly decreasing ranking functions in order to demonstrate convergence/livelock-freedom; these methods are not generally suitable for automation.

We extend the second category of techniques by exactly characterizing livelocks in a subclass of protocols on unidirectional rings. Due to Suzuki's undecidability result [7], we consider a weaker

model of computation where the processes of a unidirectional ring are *self-disabling*. A process is self-disabling if any of its execution steps disable all actions of that process until a step of its predecessor process re-enables it. A process is enabled/has an enablement if and only if its condition for taking a step is satisfied for the current value of its local variables, otherwise it is disabled. In our previous work [21], we established that a unidirectional ring propagates a fixed number of enablements along its links if and only if the ring has a livelock (Property 2). We formalize the notions of propagation and process local loops by using local binary relations on the steps of every process P_r. Theorem 1 establishes our main result: a unidirectional ring has a livelock if and only if propagations preserve the cyclic structure of local loops, for every process P_r, and there exists a global configuration from which enablement propagation starts.

Contributions. We establish an equivalence between the set of livelocks in a unidirectional ring and algebraic properties of binary relations representing local loops and local propagations of each process in the ring. The local nature of our algebraic characterization significantly simplifies reasoning about livelocks in parameterized unidirectional rings that are not necessarily uniform while avoiding exploration of the global state space. Our algebraic representation reduces design and verification of livelock-freedom to the analysis of a set of algebraic equations on compositions of binary relations. Our algebraic approach exactly characterizes all the livelocks Dijkstra's token ring [3] has, including a case where processes execute in full synchrony and the number of processes is greater than the size of the variables' domains by one.

Organization. In Section 2, we cover the necessary background, model of computation and previous results [21] about livelocks in unidirectional rings. We incrementally develop our formal framework for reasoning about livelocks and establish our main theorem in Section 3. Section 4 illustrates how we utilize our formal framework to characterize livelocks/livelock-freedom through three classical protocols on unidirectional rings. We elaborate on the related work and discuss the impact of our results in Section 5. Section 6 summarizes our contributions and discusses potential extensions. Due to space constraints, we include the proofs of Lemmas and Theorems in the Appendix.

2 Preliminaries

In this section, we present definitions of parameterized rings, computations, livelocks and a summary of our previous results about livelocks in unidirectional rings.

2.1 Parameterized Rings

A parameterized ring $p(K)$ is a triplet $\langle \Phi_p(K), \Pi_p(K), \Delta_p(K) \rangle$ where K is the number of processes and variables in the ring. $\Phi_p(K) = \{x_0, \cdots, x_{K-1}\}$ is the set of variables of the ring such that each variable x_r in $\Phi_p(K)$ has a finite domain D_r $(0 \leq r \leq K-1)$. $\Pi_p(K) = \{P_0, \cdots, P_{K-1}\}$ is the set of processes of the ring where $P_r = \langle R_r, W_r, \delta_r \rangle$ $(0 \leq r \leq K-1)$ is a triplet such that $R_r = \{x_{r\ominus 1}, x_r\}$, is the subset of variables that process P_r can read and $W_r = \{x_r\}$ is a singleton subset that includes x_r: the variable that process P_r can write. P_r (respectively, $P_{r\oplus 1}$) is the *predecessor* (*successor*) of $P_{r\oplus 1}$ (respectively, P_r).[1] At no loss of generality, our model captures the case where $|W_r| > 1$ by replacing the variables in W_r with a new variable x_r whose domain D_r is the Cartesian product of the domains of each variable in W_r.

A *local state* s^r of P_r is a valuation of the variables in R_r $(0 \leq r \leq K-1)$. The *local state space* S^r is the set of all possible local states of P_r. A *local state predicate* is any subset of S^r specified

[1] We denote modular arithmetic operations by circled operators; modular addition and subtraction are denoted by $\oplus$ and $\ominus$, respectively.

as a Boolean expression over variables of R_r. We say that a local state predicate X^r *holds in a local state* s^r denoted, $s^r \in X^r$ *if and only if* X^r evaluates to true at s^r. The value of a variable $x \in R_r$ at local state s^r is denoted $x(s^r)$. A *local transition* t^r of P_r is a pair of local states $(s^r, s^{r\prime})$ of P_r such that, : $x_{r\ominus 1}(s^r) = x_{r\ominus 1}(s^{r\prime})$, where s^r and $s^{r\prime}$ are the *source* and *target* local states of t^r, respectively. δ_r denotes the set of local transitions of P_r $(0 \leq r \leq K-1)$.

Likewise, a *global state* s_g of $p(K)$ is a valuation of all variables in $\Phi_p(K)$. We similarly define the *global state space* $S_p(K)$, a *global state predicate*, the value of a variable x at s_g and a *global transition* $t_g = (s_g, s_g{}')$, where s_g and $s_g{}'$ are the *source* and *target* global states of t_g, respectively.

The *projection* $s_g \downarrow Var$ of a global state $s_g \in S_p(K)$ on a set of variables $Var \subseteq \Phi_p(K)$ is a valuation of every variable $x \in Var$ such that $x(s_g) = x(s_g \downarrow Var)$. Likewise, we define the *projection* of a global transition $(s_g, s_g{}') \downarrow Var$ as the pair $((s_g \downarrow Var), (s_g{}' \downarrow Var))$. Inversely, every local state s^r of P_r is mapped to a set of global states $g^K(s^r) = \{s_g \in S_p(K) : \forall x \in R_r : x(s_g) = x(s^r)\}$. Likewise, every local transition t^r corresponds to a *group* of global transitions $g^K(s^r, s^{r\prime}) = \{(s_g, s_g{}') \in S_p(K) \times S_p(K) : (\forall x \in R_r : x(s_g) = x(s^r) \wedge x(s_g{}') = x(s^{r\prime})) \wedge (\forall x \notin W_r : x(s_g) = x(s_g{}'))\}$. Thus, $g^K(\delta_r)$ represents the set of global transitions of P_r in $p(K)$. The set of global transitions of $p(K)$ is the union of the set of global transitions of each process P_r, i.e.; $\Delta_p(K) = \cup_{r=0}^{K-1} g^K(\delta_r)$. As such, the *transition relation* of $p(K)$ is representable as a directed graph $T_p(K)$ such that $S_p(K)$ and $\Delta_p(K)$ are its sets of vertices and arcs, respectively.

Protocol Representation. We use a variant of Dijkstra's guarded commands language [23] to represent the set of local transitions of P_r (i.e., δ_r). A guarded command (i.e., *action*) is of the form $L : grd_r \rightarrow stmt_r$, where L is an optional label, grd_r is a local predicate of P_r, and $stmt_r$ is an assignment that updates variables of W_r *atomically*. Formally, an action $grd_r \rightarrow stmt_r$ includes a set of local transitions $(s^r, s^{r\prime})$ of P_r such that grd_r holds in every local state s^r and the atomic execution of $stmt_r$ results in a local state $s^{r\prime}$. An action $grd_r \rightarrow stmt_r$ is *enabled* in a local (global) state s^r (respectively, s_g) if and only if grd_r holds at s^r (respectively, s_g). The process P_r is *enabled*/has an *enablement* in s^r (respectively, s_g) if and only if there exists an action of P_r that is enabled at s^r (respectively, s_g). A local transition $(s^r, s^{r\prime})$ is enabled in s_g if $s_g \in g^K(s^r)$. Similarly, we denote a local transition $t^r = (s^r, s^{r\prime})$ of P_r by $t^r_{(x_{r\ominus 1}(s^r))(x_r(s^r) \rightarrow x_r(s^{r\prime}))}$. We denote the set of local transitions of P_r by local transitions parameterized by values from D_r and $D_{r\ominus 1}$. For instance, the set $\tau_r = \{t^r_{(v)(v\ominus 1 \rightarrow v)} | v \in D_r\}$ is abbreviated as $t^r(v)$ since every local transition in τ_r is uniquely determined by the value of v.

2.2 Computations and Execution Semantics

A *computation* of a protocol $p(K)$ is a *maximal* walk σ in $T_p(K)$. σ is maximal in that either σ is infinite or if it is finite, then σ reaches a global state with no outgoing global transitions. We assume an interleaving semantics, where every global transition t_g of a computation σ belongs to the group of exactly one local transition; i.e., no global transition corresponds to the joint action of more than one process, and no global transition belongs to more than one group. Thus, the sequence of local transitions of σ can be obtained by projecting every t_g in σ over the corresponding R_i of its local transition $(0 \leq i \leq K-1)$.

A state predicate I is *locally conjunctive* if and only if for every K, $I(K)$ is a conjunction of K local state predicates LC_r, where LC_r specifies a local state predicate of P_r; i.e., $I(K) = \bigwedge_{r=0}^{K-1} LC_r$. Throughout this paper, we assume that I, the set of legitimate states, is locally conjunctive

A *livelock/non-progress cycle* is a computation of $p(K)$ whose corresponding walk in $T_p(K)$ is a cycle. A livelock L for I is a livelock such that every global state of L is in a state predicate $\neg I$.

Self-Disablement. A process P_r is self-disabling if the target local state of every local transition of P_r has no outgoing local transitions in P_r. Self-disablement is crucial for establishing the sufficiency of our conditions for livelock-freedom.

Note that every deadlock-free/livelock-free protocol can be transformed to a self-disabling protocol without disrupting deadlock-freedom/livelock-freedom. In other words, as far as design is concerned, we could restrict our search for deadlock-free and/or livelock-free protocols to self-disabling protocols. An impossibility of existence of a self-disabling protocol implies an impossibility for the more general class of protocols.

2.3 Properties of Livelocks in Unidirectional Rings

We summarize our previous results about livelocks in unidirectional rings [21]. For a detailed exposition and proofs, please refer to [20].

Property 1 (Enablement Propagation). Let C$=\ll c_1, \cdots, c_k, \cdots \gg$ be a sequence of global states representing a computation of a parameterized ring $p(K)$. $\forall k > 1$: **If** ($\exists j : P_j$ is *enabled* in c_k **and** P_j is *disabled* in c_{k-1}) **then** $P_{j\ominus 1}$ is enabled in c_{k-1}; i.e., $(c_{k-1}, c_k) \in g^K(\delta_{j\ominus 1})$.

The significance of Property 1 is to illustrate that in the course of a program computation, a disabled process is enabled only by the action of its predecessor. In other words, a process can only pass at most one *enablement* to its successor.

Definition 1 (Collision). *Let $p(K)$ be a parameterized ring with adjacent processes P_r and $P_{r\oplus 1}$. Let s^r and $s^{r\oplus 1}$ be local states where P_r and $P_{r\oplus 1}$ are enabled, respectively. A* collision *is an execution of any local transition of P_r enabled at s^r.*

Property 2 (Enablement Conservation in a Unidirectional Ring). Let $p(K)$ be a parameterized ring. A computation L of $p(K)$ is a livelock *if and only if* in every global state of L, the number of enabled processes is the same.

Property 2 illustrates that a livelock in a unidirectional ring propagates a constant number of enablements without collisions.

Definition 2 (Livelock Induced Precedence Relation $\prec$). *Let a livelock L be represented by a sequence of local transitions Sch$=\ll t_0, t_1, \cdots, t_{n-1} \gg$. We say t_i precedes t_j, denoted $t_i \prec t_j$ iff*

1. *the execution of t_i in P_r enables t_j in $P_{r\oplus 1}$ or,*
2. *if t_j executes in P_r, then it collides with t_i enabled in $P_{r\oplus 1}$ or,*
3. *if 1 and 2 are false, then there exists t_k in Sch such that $t_i \prec t_k$ and $t_k \prec t_j$.*

Property 3 (Precedence Relation Reduction). Let $p(K)$ be a parameterized ring. If $p(K)$ has a livelock L, for some K, whose local transitions are represented by a sequence *Sch*$=\ll t_0, t_1, \cdots, t_{n-1} \gg$ then every precedence-preserving permutation of *Sch* represents a livelock of $p(K)$.

Property 3 establishes our observation for a reduction based on an irreflexive partial order. Godefroid [24] originally introduced partial order reduction to simplify automatic verification. We accordingly reduce our search for livelocks in unidirectional rings to a search for a representative livelock that we call a *contiguous livelock*. Let L be a livelock on a unidirectional ring having $|E|$ enablements. A *contiguous livelock* C_L has a global state where $|E|$ adjacent processes are enabled. The subsequent global states of C_L are such that only the frontmost enablement in the segment of adjacent processes propagates while the remaining $|E| - 1$ enablements do not propagate. After $K - |E|$ propagations of the frontmost enablement, a new global state with $|E|$ adjacent enablements is reached. Property 4 directly follows from Property 3.

Property 4. $p(K)$ has a livelock *if and only if* $p(K)$ has a contiguous livelock.

Effect of Fairness and Execution Semantics on Livelocks. In a livelock L of a ring $p(K)$, every process participates in L; i.e., every process executes infinitely often. Therefore, every livelock is an *impartially fair* computation.[2] Moreover, Property 3 asserts that, in a livelock L in a unidirectional ring, no matter in what order independent local transitions execute, the resultant global state is the same. It follows that the simultaneous execution of two local transitions $t^{r\oplus 1}$ and t^r is equivalent to the sequential execution $\ll t^{r\oplus 1}, t^r \gg$, in $P_{r\oplus 1}$ then P_r, respectively. Thus, $T_p(K)$ also captures the behaviors of livelocks in which multiple processes execute simultaneously, provided that $|E| < K$. The case of a fully synchronous livelock; i.e., when $|E| = K$, is not captured by $T_p(K)$ since every sequential propagation of enablements has a collision. However, Property 2 still holds for a fully synchronous livelock. Thus, our results hold in a parameterized ring for the most general execution semantics and fairness assumptions.

3 Algebraic Properties of Livelocks

In this section, we establish necessary and sufficient conditions for livelock-freedom in parameterized rings in terms of binary relations on local transitions of each process of the ring. In Subsection 3.1, we define a set of binary relations (on the local transitions of processes) that we shall use in Subsection 3.2 to specify necessary and sufficient conditions for the existence of livelocks in unidirectional rings.

3.1 Binary Relations on Local Transitions

We define our binary relations on the set of local transitions of processes. We say a transition $t^i = (s_0^i, s_1^i)$ in a process P_i *partially enables* a transition $t^j = (s_0^j, s_1^j)$ in a process P_j if and only if for every variable $x \in W_i \cap R_j$, we have $x(s_1^i) = x(s_0^j)$. The idea behind partial enablement is that the complete enablement of a local transition t^r of P_r depends on x_r and $x_{r\ominus 1}$. Thus, when a process P_r executes, it may update x_r that could in turn enable its successor depending on the value of $x_{r\oplus 1}$. Using the notion of partial enablement, we relate each local transition of a process with the rest of its local transitions and with the local transitions of its successor. Formally, we define a binary relation $\mathcal{H}_r \subseteq \delta_r \times \delta_r$ that is equal to the following set of pairs of local transitions of P_r: $\mathcal{H}_r = \{((s_i^r, s_i^{r\prime}), ({s^r}_j, {s^{r\prime}}_j)) | x_r(s_i^{r\prime}) = x_r({s^r}_j)\}$.

A cycle of local transitions in $\mathcal{H}_r$ is a *pseudolivelock* whose set of pairs of local transitions defines a relation $\eta_r \subseteq \mathcal{H}_r$. We denote by $L_p^r \subseteq \delta_r$ the subset of transitions of P_r that participate in the pseudolivelock η_r.

Likewise, we define a *local propagation* of P_r to $P_{r\oplus 1}$ as a binary relation $\mathcal{E}_r \subseteq \delta_r \times \delta_{r\oplus 1}$ where $(t^r, t^{r\oplus 1}) \in \mathcal{E}_r$ if and only if t^r partially enables $t^{r\oplus 1}$. $\mathcal{E}_r = \{((s^r, s^{r\prime}), (s^{r\oplus 1}, s^{r\oplus 1\prime})) | x_r(s^{r\prime}) = x_r(s^{r\oplus 1})\}$. A *feasible propagation* of the transitions of a pseudolivelock η_r of P_r to the transitions of a pseudolivelock $\eta_{r\oplus 1}$ is a local propagation such that $\epsilon_r \subseteq L_p^r \times L_p^{r\oplus 1}$ is left-total in L_p^r and onto on $L_p^{r\oplus 1}$; i.e., ϵ_r is a *total* binary relation.

Notation. Let R be a binary relation on a subset of local transitions of $p(K)$, $R(t_i)$ denotes some arbitrary transition t_j such that $(t_i, t_j) \in R$. In other words, $t_j = R(t_i)$ is a shorthand for $(t_i, t_j) \in R$. We denote relation composition from left to right by '$\circ$' and a sequence of composition of binary relations $a_x \circ \cdots \circ a_y$ by $a_{x \rightsquigarrow y}$, where composition is along a clockwise direction of the ring if and only if $x < y$. a^m denotes the composition of m copies of a; i.e., exponentiation. a^{-1} denotes the inverse relation of a. e denotes the *identity* binary relation; i.e., $e = \{(x, x) \in A \times A\}$, for some set A.

[2] A computation C is impartially fair if every process infinitely often participates in C.

Example 1 (3-valued Agreement).

Consider the local transitions δ_r of a 3-valued agreement protocol defined as follows for $0 \leq r \leq K-1$.

$$
\begin{array}{l}
t^r_{(2)(0\to 2)} : x_{r\ominus 1} = 2 \wedge x_r = 0 \longrightarrow x_r := 2 \\
t^r_{(2)(1\to 2)} : x_{r\ominus 1} = 2 \wedge x_r = 1 \longrightarrow x_r := 2 \\
t^r_{(1)(0\to 1)} : x_{r\ominus 1} = 1 \wedge x_r = 0 \longrightarrow x_r := 1 \\
t^r_{(1)(2\to 1)} : x_{r\ominus 1} = 1 \wedge x_r = 2 \longrightarrow x_r := 1 \\
t^r_{(0)(2\to 0)} : x_{r\ominus 1} = 0 \wedge x_r = 2 \longrightarrow x_r := 0 \\
t^r_{(0)(1\to 0)} : x_{r\ominus 1} = 0 \wedge x_r = 1 \longrightarrow x_r := 0
\end{array}
$$

We summarize the local transitions of the protocol by the set of parameterized transitions: $t^r_{(v)(v\oplus 1\to v)}$ and $t^r_{(v\oplus 1)(v\to v\oplus 1)}$.

Figure 1 illustrates $\mathcal{H}_r$ of the local transitions in Example 1. Every related pair of local transitions satisfies our definition. For instance, $(t^r_{(1)(2\to 1)}, t^r_{(2)(1\to 2)}) \in \mathcal{H}_r$ since $t^r_{(1)(2\to 1)}$ sets x_r to 1 while the local source state of $t^r_{(2)(1\to 2)}$ is partially enabled at $x_r = 1$. We apply the same reasoning to every pair of local transitions in δ_r in order to build $\mathcal{H}_r$. Using a fixpoint computation on $\mathcal{H}_r$, we determine that in this case $\eta_r = \mathcal{H}_r$ and $L^r_p = \delta_r$. For the 3-valued agreement, η_r is a strongly connected component that can be decomposed into the union of the following two bijections: η_{r1} and η_{r2}. $\eta_{r1}(t^r_{(v)(v\oplus 1\to v)}) = t^r_{(v\ominus 1)(v\to v\ominus 1)}$, $\eta_{r1}(t^r_{(v)(v\ominus 1\to v)}) = t^r_{(v\oplus 1)(v\to v\oplus 1)}$, $\eta_{r2}(t^r_{(v)(v\oplus 1\to v)}) = t^r_{(v\oplus 1)(v\to v\oplus 1)}$ and $\eta_{r2}(t^r_{(v\oplus 1)(v\to v\oplus 1)}) = t^r_{(v)(v\oplus 1\to v)}$.

Figure 2 depicts $\mathcal{E}_r$ of the local transitions in Example 1. Every connected pair of local transitions satisfies our definition of $\mathcal{E}_r$. For instance, $(t^r_{(1)(2\to 1)}, t^{r\oplus 1}_{(1)(0\to 1)}) \in \mathcal{E}_r$ since the local source state of $t^{r\oplus 1}_{(1)(0\to 1)}$ and the local target state of $t^r_{(1)(2\to 1)}$ share the value of $x_r = 1$. We apply the same reasoning to every pair of local transitions in $\delta_r \times \delta_{r\oplus 1}$ to build $\mathcal{E}_r$. A potential feasible propagation for the transitions of a given pseudolivelock $\eta_r \in \mathcal{H}_r$ is a total binary relation on $L^r_p \times L^{r\oplus 1}_p$.

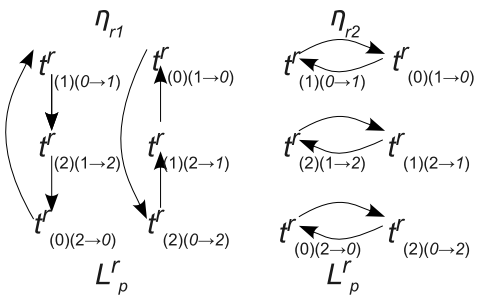

Fig. 1. Pseudolivelocks Over Local Transitions of a 3-Agreement Protocol

We decompose $\mathcal{E}_r$ into two disjoint bijections $\epsilon_{r1} : L^r_p \to L^{r\oplus 1}_p$ and $\epsilon_{r2} : L^r_p \to L^{r\oplus 1}_p$.

- $\epsilon_{r1}(t^r_{(v)(v\oplus 1\to v)}) = t^{r\oplus 1}_{(v)(v\oplus 1\to v)}$,
- $\epsilon_{r1}(t^r_{(v)(v\ominus 1\to v)}) = t^{r\oplus 1}_{(v)(v\ominus 1\to v)}$,
- $\epsilon_{r2}(t^r_{(v)(v\oplus 1\to v)}) = t^{r\oplus 1}_{(v)(v\ominus 1\to v)}$, and
- $\epsilon_{r2}(t^r_{(v)(v\ominus 1\to v)}) = t^{r\oplus 1}_{(v)(v\oplus 1\to v)}$.

Figure 2 captures the decomposition of ϵ_r into ϵ_{r1} and ϵ_{r2}. $\triangleleft$

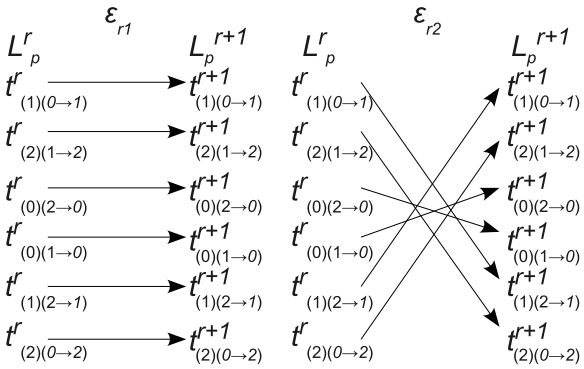

Fig. 2. Feasible Propagations Over Local Transitions of a 3-Agreement Protocol

3.2 Conditions for Livelock-Freedom in Unidirectional Rings

We incrementally establish a set of necessary and sufficient conditions for livelock-freedom in unidirectional rings.

Lemma 1 (Existence of a pseudolivelock). *If $p(K)$ has a livelock L, then for every process P_r there exists a pseudolivelock $\eta_r \subseteq \mathcal{H}_r$ strongly connecting L_p^r.*

Proof. Let $p(K)$ have a livelock L. Lemma 1 implies that every P_r participates in L. Project L on x_r to obtain the recurrent sequence of local transition $\sigma_r = \ll t_0^r, t_1^r, \cdots, t_i^r, t_{i\oplus1}^r, \cdots, t_{m-1}^r \gg^*$ of P_r. As such, every t_i^r in σ_r is infinitely often enabled, thus, for all $0 \leq i \leq m-1 : t_i^r$ partially enables $t_{i\oplus1}^r$ through the update of x_r. Therefore, there exists a cycle $\eta_r = \{(t_i^r, t_{i\oplus1}^r)\} \subseteq \mathcal{H}_r$ and η_r strongly connects $L_p^r = \{t_i^r\}$, where L_p^r is the set of local transitions in σ_r. Hence, $\eta_r \subseteq \mathcal{H}_r$ is a pseudolivelock that strongly connects L_p^r, for every P_r.

Lemma 2 asserts the existence of feasible propagations $\epsilon_r \subseteq (L_p^r \times L_p^{r\oplus1}) \cap \mathcal{E}_r$ for every P_r in a livelock of $p(K)$.

Lemma 2 (Existence of a feasible propagation). *If $p(K)$ has a livelock L, then for every P_r there exists a feasible propagation designated by a total binary relation $\epsilon_r \subseteq (L_p^r \times L_p^{r\oplus1})$ where L_p^r is the set of local transitions of some pseudolivelock generated by L in P_r.*

Proof. Since $p(K)$ has a livelock L, every P_r has a pseudolivelock η_r whose set of local transitions is designated by L_p^r (Lemma 1). In L, for every P_r, the execution of every local transition in L_p^r passes an enablement to $P_{r\oplus1}$ as implied by Properties 1 and 2. As such, the execution of every $t^r \in L_p^r$ partially enables a transition $t^{r\oplus1} \in L_p^{r\oplus1}$, for every $0 \leq r \leq K-1$. Otherwise, we have two cases: (1) there exists a local transition in L_p^r that does not enable any transition in $L_p^{r\oplus1}$, thereby not propagating an enablement to $P_{r\oplus1}$ or, (2) there exists a local transition in $L_p^{r\oplus1}$ that is not enabled by any transition in L_p^r, thereby blocking the pseudolivelock $\eta_{r\oplus1}$. Both cases contradict the existence of L. Therefore, ϵ_r is a total binary relation consisting of the set of pairs $(t^r, t^{r\oplus1}) \in L_p^r \times L_p^{r\oplus1}$ such that the execution of t^r partially enables $t^{r\oplus1}$; i.e., $\epsilon_r \subseteq ((L_p^r \times L_p^{r\oplus1}) \cap \mathcal{E}_r)$.

For simplicity of presentation, we define the binary relation $\kappa_r \subseteq L_p^r \times L_p^{r\ominus1}$ linking two local transitions $(t^r, t^{r\ominus1}) \in L_p^r \times L^{r\ominus1}$ if and only if there exists $t_p^{r\ominus1} \in L_p^{r\ominus1}$ such that $t_p^{r\ominus1}$ partially enables $t^{r\ominus1}$ in the pseudolivelock $\eta_{r\ominus1}$ and $t_p^{r\ominus1}$ partially enables t^r in the feasible propagation $\epsilon_{r\ominus1}$. Definition 3 formalizes this notion.

Definition 3 (Contiguousness). $\kappa_r = \epsilon_{r\ominus 1}^{-1} \circ \eta_{r\ominus 1}$.

We establish our necessary and sufficient conditions for the existence of livelocks in the following Theorem.

Theorem 1. *$p(K)$ has a livelock L* if and only if *for every P_r, there exists a pseudolivelock $\eta_r \subseteq \mathcal{H}_r$, strongly connecting L_p^r, and a feasible propagation $\epsilon_r : L_p^r \to L_p^{r\oplus 1}$ that satisfy the following two equations:*

1. Ring Reconstruction Equation: $\exists |E| > 0 : \kappa_{r\rightsquigarrow r-|E|+1} = \left\{ \begin{array}{ll} \epsilon_{r\rightsquigarrow r+K-|E|-1} & |E| < K \\ e & |E| = K \end{array} \right\}$, *and*
2. Enablement Flow Equation: $\eta_r \circ \epsilon_r = \epsilon_r \circ \eta_{r\oplus 1}$.

Proof. Necessity of the Conditions for the Existence of Livelocks: Since $p(K)$ has a livelock L, it follows from Lemmas 1 and 2 that for every P_r, there exist a pseudolivelock $\eta_r : L_p^r \to L_p^r$ and a feasible propagation $\epsilon_r : L_p^r \to L_p^{r\oplus 1}$.

1. *Ring Reconstruction Equation.* Consider a livelock L with $|E|$ ($|E| > 0$) enablements in the ring of size K. There exists a contiguous livelock C_L that preserves the partial order imposed on the local transitions of L (Lemma 4). Consider a global state of C_L where $|E|$ enablements are adjacent in processes $P_{r-|E|+1}$ to P_r, and t_j^r is enabled in P_r. A propagation of the rightmost enablement in C_L is a sequence of $K - |E|$ propagations from P_r to $P_{r+K-|E|-1}$. By definition of ϵ_r and κ_r, the enabled sequence of local transitions along the direction of propagation of the rightmost enablement of C_L is $\ll t_j^r,\ \epsilon_r(t_j^r),\ (\epsilon_r \circ \epsilon_{r\oplus 1})(t_j^r),\ \cdots,\ \epsilon_{r\rightsquigarrow(r+K-|E|-1)}(t_j^r) \gg$; where $t_j^{r\oplus K-|E|-1} = \epsilon_{r\rightsquigarrow(r+K-|E|-1)}(t_j^r)$. On the other hand, going anticlockwise along the contiguous sequence of enabled local transitions reached after $K - |E|$ propagations yields the same $t_j^{r\oplus K-|E|-1} = \kappa_{r\rightsquigarrow(r-|E|+1)}(t_j^r)$. The Ring Reconstruction Equation follows immediately.
2. *Flow Equation.* For some arbitrary P_r, let $t_j^r \in L_p^r$ be an arbitrary enabled local transition. Since L is a livelock, every local transition in L_p^r eventually executes in the sequence projected from L on x_r and captured by η_r; i.e., $\sigma_r = \ll t_j^r, \eta_r(t_j^r), (\eta_r \circ \eta_r)(t_j^r), \cdots, \eta_r^{m_r-1}(t_j^r) \gg^*$. Since every local transition of L_p^r is self-disabling, σ_r can execute if and only if every local transition in σ_r gets enabled by some local transition $t_j^{r\ominus 1} \in \sigma_{r\ominus 1}$. In fact, the enablement of every t_j^r in a unidirectional ring depends only on the values of $x_{r\ominus 1}$ and x_r, i.e.; can only be enabled by the execution of $\eta_r^{-1}(t_j^r) = t_{j\ominus 1}^r$ and $\epsilon_{r\ominus 1}^{-1}(t_j^r) = t_j^{r\ominus 1}$. Note that local transitions as illustrated in Figure 4 are not all necessarily distinct, in fact m is the least common multiple of m_r and $m_{r\oplus 1}$. In L, $t_{j\ominus 1}^r \prec t_j^{r\ominus 1}$ since the execution of $t_j^{r\ominus 1}$ either (1) collides with $t_{j\ominus 1}^r$ if $|E| > 1$ or (2) transitively depends on t_j^r due to a consecutive propagation on the ring if $|E| = 1$; i.e., $t_j^{r\ominus 1} = \epsilon_{r\rightsquigarrow r+K-1}(t_{j\ominus 1}^r)$. In case (1), clearly $t^{r\ominus 1} = \kappa_r(t_{j\ominus 1}^r)$, while in case (2), using the Reconstruction Equation, we reach the same result. Thus, $\kappa_r(t_{j\ominus 1}^r) = t_j^{r\ominus 1}$ for $|E| > 0$ (Figure 3). Combining identities to obtain relations only on t_j^r, we obtain $(\kappa_r \circ \epsilon_{r\ominus 1})(t_j^r) = \eta_r(t_j^r)$. By substitution for κ_r from Definition 3, we obtain the Enablement Flow Equation.

Sufficiency of the Conditions for the Existence of Livelocks: We demonstrate that, if for every P_r, there exists η_r and ϵ_r satisfying the Flow and the Ring Reconstruction Equations (κ_r is redundant according to Definition 3), then $p(K)$ has a contiguous livelock with $|E|$ enablements. We construct a livelock with $|E|$ enablements as follows:

Initial State. Let $t^r \in L_p^r$ be a local transition in the pseudolivelock η_r whose cyclic structure is preserved by the feasible propagation ϵ_r, where $0 \leq r \leq K-1$. Since the Reconstruction Equation holds for some $|E|$ and K, we construct a global state s_I of $p(K)$ with $|E|$ enabled local transitions $t^r = (s^r, s^{r\prime})$ in L_p^r where $0 \leq r \leq |E|-1$, $t^{|E|-1}$ being the rightmost enabled local transition.

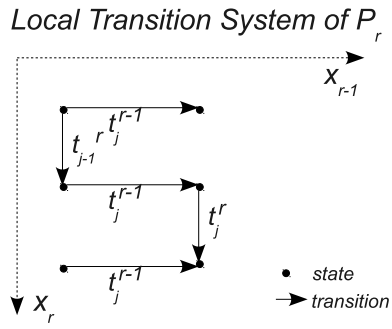

Fig. 3. The source state of t_i^r and $t_j^{r\ominus 1}$ share the value of $x_{r\ominus 1}$, that is why $\kappa_r(t_i^r) = t_j^{r\ominus 1}$ for any $|E| > 0$
.

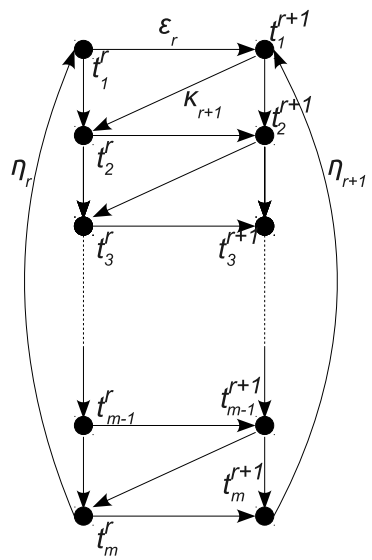

Fig. 4. ϵ_r preserves the cyclic structure of η_r and $\eta_{r\oplus 1}$.

For each enabled P_r $(0 \leq r \leq |E| - 1)$ in s_I, let $t^r = \kappa_{r\oplus 1}(t^{r\oplus 1})$. Each disabled process P_r, $|E| \leq r \leq K - 1$ is in the target local state $s^{r\prime}$ of its corresponding disabled local transition t^r. For each disabled P_r $(|E| \leq r \leq K - 1)$ in s_I, let $t^{r\oplus 1} = \epsilon_r(t^r)$. It is easy to verify that the Reconstruction Equation holds for $t^{|E|}$.

Execution. Substituting the result of Definition 3 into the Flow Equation, we obtain an equivalent form of the Flow Equation where $\kappa_r \circ \epsilon_{r\ominus 1} = \eta_r$. We use structural induction to demonstrate that every disabled P_r eventually gets enabled provided that the Flow Equation holds at every process.

- *Base Case.* We demonstrate that the execution of $t^{|E|-1}$ enables $P_{|E|}$. We apply the latter form of the Flow Equation at $P_{|E|}$ in the initial global state s_I to obtain that $(\kappa_{|E|} \circ \epsilon_{|E|-1})(t^{|E|}) = \eta_{|E|}(t^{|E|})$. Substituting the value of $\kappa_{|E|}(t^{|E|})$ from s_I, we obtain that $\epsilon_{|E|-1}(t^{|E|-1}) = \eta_{|E|}(t^{|E|})$. However, since $t^{|E|}$ already executed in $P_{|E|}$; i.e, is disabled in target local state $s^{|E|\prime}$, $\eta_{|E|}(t^{|E|})$ is partially enabled by $t^{|E|}$ and the execution of $t_j^{|E|-1}$ fully enables $\eta_{|E|}(t^{|E|})$ in $P_{|E|}$.
- *Induction Hypothesis.* The execution of every disabled process P_q for $|E| \leq q < r$ enables $P_{q\oplus 1}$.

- *Induction Step.* P_r is enabled in $\eta_r(t^r)$, $P_{r\oplus 1}$ is disabled in $s^{r\oplus 1'}$ and $t^{r\oplus 1}$ partially enables $\eta_{r\oplus 1}(t^{r\oplus 1})$ in $P_{r\oplus 1}$. In s_I, $t^{r\oplus 1} = \epsilon_r(t^r)$, therefore, (1) $t^{r\oplus 1}$ partially enables $(\epsilon_r \circ \eta_{r\oplus 1})(t^r)$. In P_r, (2) $\eta_r(t^r)$ partially enables $(\eta_r \circ \epsilon_r)(t^r)$. Since the Flow Equation holds at P_r, we conclude from (1) and (2) that $\eta_r(t^r)$ and $t^{r\oplus 1}$ partially enable the same local transition $\eta_{r\oplus 1}(t^{r\oplus 1})$. By the induction hypothesis, $\eta_r(t^r)$ is enabled in P_r, and $\eta_{r\oplus 1}(t^{r\oplus 1})$ is partially enabled by the executed $t^{r\oplus 1}$. Thus, the execution of $\eta_r(t^r)$ in P_r fully enables $\eta_{r\oplus 1}(t^{r\oplus 1})$.
- *Terminal Case.* At P_{K-1}, applying the Flow Equation yields $\epsilon_{K-1}(\eta_{K-1}(t^{K-1})) = \eta_K(t^0)$. However, the execution of t^{K-1} does not enable $\eta_K(t^0)$ and disables t^0 that has not executed yet, thereby contradicting the Flow Equation. Thus, no collision occurs at P_{K-1} with P_0.

This proves that as long as the Flow Equation holds, collisions do not occur and propagation always occurs for every P_r.

By repeating the similar structural induction for every propagation of the rightmost enablement, and since η_r is a cycle, we construct a contiguous livelock C_L of $p(K)$ having $|E|$ enablements. Thus follows from Lemma 4 that $p(K)$ has a livelock L.

The local nature of pseudolivelocks and feasible propagations allows us to exactly characterize livelocks in a unidirectional ring for arbitrary K. Intuitively, the Flow Equation asserts that $\epsilon_r \subseteq L_p^r \times L_p^{r\oplus 1}$ is a *cyclic structure-preserving* total binary relation. Given an initial global state from which pseudolivelocks could start their execution by satisfying the Reconstruction Equation, the Flow Equation guarantees a seamless propagation of enablements for every process P_r. Therefore, a livelock exists. Conversely, the existence of a livelock L under the assumption of self-disablement imposes an order on the execution of enabled local transitions in adjacent processes to avoid collisions (Lemma 3). As such, the structure of pseudolivelocks, whose connectedness is preserved by feasible propagations, is the only representation for L.

Example 2 (3-valued Agreement).

Consider the pseudolivelock represented by $\mathcal{H}_r = \eta_{r1} \cup \eta_{r2}$ as in Figure 1. Consider $\mathcal{E}_r = \epsilon_{r1} \cup \epsilon_{r2}$ as depicted in Figure 2. For simplicity of presentation, we check for potential livelocks formed by η_{r1} and ϵ_{r1} restricted to $L_p^r = \{t^r_{(v\oplus 1)(v\to v\oplus 1)} | 0 \leq v \leq 2\}$.

In Figure 1, $\eta_{r1}(t^r(v)) = t^r(v \oplus 1)$. Figure 2 illustrates that $\epsilon_{r1}(t^r(v)) = t^{r\oplus 1}(v)$. Checking the Flow Equation, we obtain $(\eta_{r1} \circ \epsilon_{r1})(t^r(v)) = \epsilon_{r1}(t^r(v \oplus 1)) = t^{r\oplus 1}(v \oplus 1)$. The right hand side of the Flow Equation evaluates to $(\epsilon_{r1} \circ \eta_{(r\oplus 1)_1})(t^r(v)) = \eta_{(r\oplus 1)_1}(t^{r\oplus 1}(v)) = t^{r\oplus 1}(v \oplus 1)$. Therefore, the Enablement Flow Equation holds for η_{r1} as a pseudolivelock and ϵ_r^1 as a feasible propagation. Since our derivation is independent of r, the Flow Equation holds for every P_r. Note that $\eta_{r1} \circ \epsilon_{r2} \neq \epsilon_{r2} \circ \eta_{(r\oplus 1)_2}$; i.e., ϵ_{r2} does not preserve the cylic structure of η_{r1} in $\eta_{(r\oplus 1)_2}$.

To apply the Reconstruction Equation, we evaluate $\kappa_r(t^r(v)) = (\epsilon^{-1}_{r\ominus 1} \circ \eta_{r\ominus 1})(t^r(v)) = t^{r\ominus 1}(v \oplus 1)$. We substitute $t^r(v)$ into the Reconstruction Equation to obtain $t^{r-|E|}(v \oplus |E|) = t^{r-|E|}(v)$. In other words, $|E| = 0$ in modulo 3; i.e., $|E|$ is a multiple of 3 regardless of the value of K. Thus, 3-valued agreement has a livelock formed by local transitions in L_p^r. This completes our proof. $\triangleleft$

Notice that, using the Reconstruction Equation at some local transition $t_j^r \in L_p^r$, we can form an initial global state of a contiguous livelock in $p(K)$ with $|E|$ enabled processes. The universally quantified version of the Reconstruction Equation follows directly from the iterative application of the Flow Equation to the initial global state. In this way, the Reconstruction Equation needs to hold only for some process P_r and at some local transition in L_p^r.

Example 3 (6-valued Agreement).

To appreciate the independence of the Flow and Reconstruction Equations, consider a 6-valued agreement protocol where $K = 4$ and consisting of transitions in $\delta_r = L_p^r = \{t^r_{(v\oplus 1)(v\to v\oplus 1)} | 0 \leq v \leq 5\}$. Despite the fact that the Flow Equation holds at every P_r, using a similar reasoning to the one in Example 2, the Ring Reconstruction Equation holds only for $|E| = 0$ when $K = 4$. In fact, substitution in the reconstruction equation yields that $|E|$ is a multiple of 6. This demonstrates that 6-valued agreement is livelock free for $2 \leq K \leq 5$. $\triangleleft$

Corollary 1 establishes another form of the Ring Reconstruction Equation.

Corollary 1 (Calculation of $|E|$). *If $p(K)$ has a livelock L with $|E|$ enablements, then $\eta_r^{|E|} = \epsilon_{r\rightsquigarrow r+K-1}$, for every process P_r.*

Proof. We present a sketch of the proof. We advise the reader to workout the details of the algebra as an exercise. Theorem 1 implies the Flow and Reconstruction Equations. Substitute for κ_r from Definition 3 in the Ring Reconstruction Equation to obtain an expression in terms of η_r and ϵ_r. Right and left-compose the Flow Equation by ${\epsilon_r}^{-1}$. The result allows to drag left all of η_r in the Reconstruction Equation and the lemma follows.

Corollary 1 is not sufficient for proving the existence of livelocks. To illustrate that, we consider a Sum-Not-Two Example.

Example 4 (Sum-Not-Two).

The Sum-Not-Two protocol has variables' domains $D_r = \{0, 1, 2\}$. The set of legitimate states is such that, for every P_r, $x_r + x_{r\ominus 1} \neq 2$.

The set of local transitions is $\delta_r = \{t^r_{(v)((2\ominus v)\to(3\ominus v))} | v \in D_r\}$. Thus, δ_r has only three local transitions in a pseudolivelock; i.e., $L_r^p = \delta_r$. Since the transitions of δ_r are fully determined by v, we use the shorthand $t^r(v)$ for $t^r_{(v)((2\ominus v)\to(3\ominus v))}$. $\eta_r(t^r(v)) = t^r_{(v\ominus 1)((3\ominus v)\to(1\ominus v))} = t^r(v \ominus 1)$. $\epsilon_r(t^r(v)) = t^{r\oplus 1}(3 \ominus v)$. The left hand side of the Flow Equation evaluates to $t^{r\oplus 1}(1 \ominus v)$ while the right hand side evaluates to $t^{r\oplus 1}(2 \ominus v)$. Thus, the Flow Equation does not hold and $p(K)$ is livelock-free.

On the other hand, applying Corollary 1 generates a pair of values for K and $|E|$ where K is even and $|E|$ is a multiple of 3. Consequently, Corollary 1 provides a necessary but insufficient condition for the existence of livelocks. $\triangleleft$

Example 5 (l-valued Agreement).

Consider a general agreement protocol where $D_r = \{0, 1, \cdots, l-1\}$ for all r. The set of local legitimate states of P_r is captured by $x_{r\ominus 1} = x_r$. Consider the following candidate protocol.

$$t^r(v, w) : (x_{r\ominus 1} = w) \wedge (x_r = v) \to x_r := w$$

where $v, w \in D_r$ are parameters. We demonstrate how to determine what type of constraints are put on $t^r_{(v,w)}$.

By definition of η_r, we get $\eta_r(t^r(v, w)) = t^r(w, y)$, where $y \in D_r$. Applying ϵ_r to $t^r(w, y)$ generates $t^{r\oplus 1}(z, y)$. Computing the right hand side of the Flow Equation in a similar way, we obtain $\epsilon_r(t^r(v, w)) = t^{r\oplus 1}(y_1, w)$ and $\eta_{r\oplus 1}(t^{r\oplus 1}(y_1, w)) = t^{r\oplus 1}(w, z_1)$. The Flow Equation holds if and only if $w = z$ and $y = z_1$. Thus, $\epsilon_r(t^r(v, w)) = t^{r\oplus 1}(v, w)$. In other words, the Flow Equation holds for an $l-1$ Agreement protocol if and only if a local transition in an arbitrary pseudolivelock of P_r enable exactly the same local transition in $P_{r\oplus 1}$.

Let m denote the length of a sequence of local transitions that repeat inside a pseudolivelock; i.e., m is the multiplicity/periodicity of η_r. The calculation of $|E|$ according to Corollary 1 yields $\eta_r^{|E|}(t^r(v,w)) = t^r(v,w)$, this is possible if and only if $|E|$ is a multiple of m.

As such, we exactly characterize the set of possible livelocks in an $l-1$ agreement protocol of arbitrary size. We conclude that breaking symmetry guarantees livelock-freedom for a general agreement protocol. $\triangleleft$

4 Additional Examples

In this section, we examine three classical protocols on a unidirectional ring to exactly determine their livelocks.

Example 6 (l-coloring).

A general parameterized set of local transitions for an l-coloring protocol has the form: $t^r(v,w) : (v)(v \to w)$, where $v \neq w$. We start by applying the Flow Equation to $t^r(v,w)$. $\eta_r(t^r(v,w)) = t^r(w,y)$ and $\epsilon_r(t^r(w,y)) = t^{r\oplus 1}(y,z)$. $\epsilon_r(t^r(v,w)) = t^{r\oplus 1}(w,y_1)$ and $\eta_{r\oplus 1}(t^{r\oplus 1}(w,y_1)) = t^{r\oplus 1}(y_1,z_1)$. The Flow Equation holds if and only if $y_1 = y$ and $z_1 = z$. Substituting into the definitions of η_r and ϵ_r, we deduce that $\epsilon_r = \eta_r \circ \mathit{shift}_1 = \mathit{shift}_1 \circ \eta_{r\oplus 1}$, where $\mathit{shift}_k(t^r(v)) = t^{r\oplus k}(v)$. Clearly, $\mathit{shift}_K = \mathit{shift}_1^K = e$. We identify this commutativity property as a *shift invariance* property.

By substitution into the result of Corollary 1 and reuse of the shift invariance property, we obtain that $\eta_r^{|E|} = \eta_r^K$. Thus follows that $K - |E|$ is a multiple of m, where m is the length of the cycle of pseudolivelock η_r. Note that shift invariance holds if and only if L_p^r are all symmetric; i.e., have the same set of transitions upto shifts.

Since every local state of P_r where $x_r = x_{r\ominus 1}$ should have an outgoing local transition in a self-stabilizing coloring protocol [20], our result implies an impossibility of self-stabilizing symmetric l-coloring over unidirectional rings.

Example 7 (Sum-Not-($l-1$)).

In a Sum-Not-($l-1$) protocol, a legitimate local state is such that $x_{r\ominus 1} + x_r \neq l-1$, where $D_r = \{0, 1, \cdots, l-1\}$. A general parameterized action detects whether the sum of $x_{r\ominus 1}$ and x_r is $l-1$ and updates the value of x_r accordingly. We study the conditions under which livelocks form in Sum-Not-($l-1$).

A parameterized local transition of P_r has the form $t^r(v) : (l-1 \ominus v)(v \to f^r(v))$, where $f^r : D_r \to D_r$ is an arbitrary bijection. Applying the Flow Equation on $t^r(v)$, $(\eta_r \circ \epsilon_r)(t^r(v)) = \epsilon_r(t^r(f^r(v))) = t^{r\oplus 1}(l-1 \ominus f^r(v))$. $(\epsilon_r \circ \eta_{r\oplus 1})(t^r(v)) = \eta_{r\oplus 1}(t^{r\oplus 1}(l-1\ominus v)) = t^{r\oplus 1}(f^{r\oplus 1}(l-1\ominus v))$. Hence, the Flow Equation holds if and only if $f^{r\oplus 1}(l-1\ominus v)\oplus f^r(v) = l-1$. Choosing $f^r(v) = v \oplus i_r$ for every r; the Flow Equation reduces to $l-1\ominus(v\oplus i_r) = (l-1\ominus v)\oplus i_{r\oplus 1}$. This holds only when $i_r \oplus i_{r\oplus 1} = 0$ in modulo l. In fact, choosing $i_r = i_{r\oplus 1} \neq l/2$ guarantees livelock freedom.

Example 8 (Dijkstra's Token Ring).

Dijkstra's token ring [3] is a mutual exclusion protocol over a unidirectional ring where $D_r = \{0, \cdots, l-1\}$. A legitimate global state of the ring is such that $|E| = 1$. Dijkstra provides a protocol where δ_0 has the parameterized local transition $t^0(v) : (v)(v \to v\oplus 1)$ and δ_i $(1 \leq i \leq K-1)$ consists of the local transitions $t^i(v,w) : (w)(v \to w)$, $(v \neq w)$. We illustrate a proof of livelock-freedom for $|E| > 1$ when $l \geq K-1$ by using the Flow Equation in Theorem 1 and Corollary 1.

At P_0, we apply the Flow Equation to obtain, $\eta_0(t^0(v)) = t^0(v\oplus 1)$ and $\epsilon_0(t^0(v\oplus 1)) = t^1(w, v\oplus 2)$. $\epsilon_0(t^0(v)) = t^1(w_1, v\oplus 1)$ and $\eta_1(t^1(w_1, v\oplus 1)) = t^r(v\oplus 1, z_1)$. The Flow Equation holds if and only

if $w = v \oplus 1$ and $z_1 = v \oplus 2$. Thus, $\epsilon_0(t^0(v)) = t^1(v, v \oplus 1)$, $w_1 = v$, $\eta_1(t^1(v, v \oplus 1)) = t^1(v \oplus 1, v \oplus 2)$ and $L_p^1 = \{t^1(v, v \oplus 1) | v \in D_1\}$.

At P_i, $1 \leq i \leq K-2$, $(\eta_i \circ \epsilon_i)(t^1(v, v \oplus 1)) = \epsilon_i(t^i(v \oplus 1, v \oplus 2)) = t^{i \oplus 1}(w, v \oplus 2)$. $(\epsilon_i \circ \eta_{i \oplus 1})(t^1(v, v \oplus 1))$ $= \eta_{i \oplus 1}(t^{i \oplus 1}(w_1, v \oplus 1)) = t^{i \oplus 1}(v \oplus 1, z_1)$. The Flow Equation holds if and only if $w = v \oplus 1$ and $z_1 = v \oplus 2$. It follows that $\epsilon_i(t^i(v, v \oplus 1)) = t^{i \oplus 1}(v, v \oplus 1)$ and $w_1 = v$. Thus, $\eta_{i \oplus 1}(t^{i \oplus 1}(v, v \oplus 1)) = t^{i \oplus 1}(v, v \oplus 1)$ and $L_p^{i \oplus 1} = \{t^{i \oplus 1}(v, v \oplus 1) | v \in D_{i \oplus 1}\}$.

So far, the feasible propagation at P_{K-1} is the only undetermined relation. We apply the Flow Equation at P_{K-1}. $(\eta_{K-1} \circ \epsilon_{K-1})(t^{K-1}(v, v \oplus 1)) = \epsilon_{K-1}(t^{K-1}(v \oplus 1, v \oplus 2)) = t^0(v \oplus 2)$. $(\epsilon_{K-1} \circ \eta_0)(t^{K-1}(v, v \oplus 1)) = \eta_0(t^0(v \oplus 1)) = t^0(v \oplus 2)$. Hence, the Flow Equation holds at every P_r.

To summarize our results, we have $\eta_r(t^r(v, v \oplus 1)) = t^r(v \oplus 1, v \oplus 2)$ for $r \neq 0$, $\eta_0(t^0(v)) = t^0(v \oplus 1)$, $\epsilon_r(t^r(v, v \oplus 1)) = t^{r \oplus 1}(v, v \oplus 1)$ for $1 \leq r \leq K-2$, $\epsilon_0(t^0(v)) = t^1(v, v \oplus 1)$ and $\epsilon_{K-1}(t^{K-1}(v, v \oplus 1))$ $= t^0(v \oplus 1)$.

Applying Corollary 1 at P_0, we obtain that $t^0(v \oplus |E|) = t^0(v \oplus 1)$; this holds if and only if $|E| - 1$ is a multiple of l: $t^0(v)$ increases v by 1 in modulo l. That is to say, $|E| = (k \times l) + 1$, where $k \in \mathbb{N}$. It follows that $|E| \in \{1, l+1, 2l+1, \cdots, kl+1, \cdots\}$; these are the values of $|E|$ for which Dijkstra's token ring has livelocks. To guarantee livelock freedom for $|E| \neq 1$, $l \geq K$ should always hold. For the case where $l = K-1$, Dijkstra's token ring has a livelock with K enablements, however, this livelock exactly occurs when all processes synchronously execute; i.e., under fully synchronous execution semantics. In other words, Dijkstra's token ring has no livelocks with $|E| > 1$ for $l \geq K-1$ assuming that all K processes can never execute together. For $l \geq K$, the protocol has no livelocks with $|E| > 1$ assuming no constraints, whatsoever, on the execution semantics. Our method detects that the only class of livelocks in Dijkstra's token ring are propagations of only one enablement: this is a legitimate behavior of Dijkstra's token ring. Since our equations capture the set of all livelocks in $p(K)$, this proves that Dijkstra's token ring has no livelocks outside $I(K)$.

5 Discussion of Related Work

We elaborate on three categories of related work. The first includes models of computation whose livelock verification is decidable. The second category captures early work about livelock characterization in parallel programs and network protocols. The third category lists representative results in static analysis of livelocks. Throughout, we discuss the relationship between our results and the presented categories.

Parameterized Verification Using Cutoff's. Emerson *et al.* [13–16] reduce subclasses of the *parameterized verification problem* to the verification of small network sizes with fixed number of processes. Emerson and Namjoshi [13] assume a ring of symmetric processes that communicate by means of a token. They demonstrate that the verification of conjunctive properties over pairs of processes reduces to verification of rings with a fixed size. Moreover, the authors demonstrate that adopting value-carrying tokens renders the verification problem undecidable. In [14], Emerson and Namjoshi extend their approach by reducing arbitrary-sized, star-interconnected, synchronous protocols to abstract graphs of finite size. Emerson and Kahlon [16] demonstrate the existence of small model reductions to parameterized valued-token passing rings as long as every process passes the tokens for a finite number of times. In [15], they extend their reduction to arbitrary sized models whose actions have conjunctive or disjunctive guards. Their model can only capture processes all of which access the same set of variables.

Our model of a unidirectional ring is incomparable to Emerson *et al.*'s. First, we do not assume symmetry. Second, our model represents processes that communicate multi-valued tokens. However, our assumption of self-disabling processes is more restrictive than Emerson *et al.*'s. Nevertheless,

a unidirectional ring with self-disabling processes captures a class of useful distributed protocols that is not covered by the aforementioned methods.

Livelocks in Network Protocols. In one of the early investigations of livelocks, Kwong [1, 8] characterizes a livelock in a parallel program by an infinite computation where some process is never executed; a.k.a., starvation. Kwong demonstrates two techniques for proving livelock-freedom: (1) Sufficient conditions that guarantee the absence of livelocks and, (2) a well-founded set technique. Kwong's first method is amenable to automatic verification, however, the sufficient conditions he establishes are applicable only to deterministic protocols, with non-interfering[3] and independent local transitions. Gouda and Chang [9] identify a subset of networks that can be exactly abstracted by finite graphs and thus, whose livelock-freedom is decidable. Despite the fact that Gouda and Chang [9] capture unbounded channels, their approach is not applicable for our purpose of proving livelock-freedom in a parameterized ring.

Static Analysis of Livelocks Leue *et al.* [25, 26, 10] devise the notion of cycle dependency for control flow analysis. Intuitively, a cycle dependency is a relation between local cycles C_1 and C_2 such that C_2 continues to execute if C_1 is executed repeatedly. The authors demonstrate a sufficient test for livelock-freedom by transforming the input protocol to an integer linear program. Their knowledge of cycle dependencies often results in better scaling of their exploration for livelocks. Ouaknine *et al.* [11] provide a sound but incomplete syntactic check on CSP[4] protocols for livelock-freedom. Blieberger *et al.* [12] conservatively check livelock-freedom in Ada programs by examining the communication patterns between potential loops. In a sense, our feasible propagation is a formalization of the notion of dependency between local cycles captured by Leue *et al.* and Blieberger *et al.* [26, 12], respectively.

Discussion. Our algebraic characterization is necessary and sufficient for livelocks in unidirectional rings whose processes are self-disabling. In spite of our restricted model of computation, we still capture many protocols of interest that have self-disabling processes. A relaxation of self-disablement renders our characterization only necessary but insufficient for livelock-freedom. A more powerful model of computation on a unidirectional ring converts livelock-freedom verification to an undecidable problem. As such, an acceptable characterization of livelocks in more general models of computation (as in Suzuki [7]) essentially has non-identical necessary and sufficient conditions for livelock-freedom, respectively.

6 Conclusions

Livelocks are among the most intricate concurrency flaws in distributed algorithms. Verification of livelocks in arbitrary-sized rings is generally undecidable [7]. We devised an exact characterization of livelocks in a subclass of protocols on unidirectional rings whose processes are self-disabling; i.e., the execution of an action of some process disables all the actions of that process. Our characterization reduced verification and design of livelock-freedom to the solution of equations on compositions of binary relations representing processes local looping: pseudolivelocks, and dependency among local loops: feasible propagations, respectively. Due to the local nature of our binary relations, our characterization is independent of the number of processes in the network, thereby circumventing global state space exploration. We illustrated the feasibility of our mathematical model through four examples: agreement, l-coloring, sum-not-$(l-1)$ and the classical Dijktra's token-ring [3].

One major application of our framework is in the automatic design and verification of convergence [5]. By combining methods for local reasoning about deadlocks [21] with our algebraic

[3] Non-interfering local transitions do not disable each other
[4] Communicating Sequential Processes

characterization of livelocks, we shall investigate sound and complete algorithms – that hinder altogether state explosion – for the automatic synthesis of convergence in unidirectional rings. We plan to generalize our current results to a wider class of protocols in two directions: (1) we shall consider arbitrary network topologies with self-disabling processes, (2) we shall investigate unidirectional rings whose processes are not necessarily self-disabling. In the first case, our current characterization is sufficient for proving the existence of livelocks in a sub-ring of the arbitrary network. In the second case, the undecidability of the problem enforces us to study non-identical necessary and sufficient conditions for livelock-freedom.

References

1. Y. Kwong, "Livelocks in parallel programs," *International Journal of Computer Mathematics*, vol. 10, no. 2, pp. 121–135, 1981.
2. L. Lamport, "A new solution of dijkstra's concurrent programming problem," *Communications of the ACM*, vol. 17, no. 8, pp. 453–455, 1974.
3. E. W. Dijkstra, "Self-stabilizing systems in spite of distributed control," *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
4. F. Gartner, "Fundamentals of fault-tolerant distributed computing in asynchronous environments," *ACM Computing Surveys*, vol. 31, no. 1, 1999.
5. A. Arora and M. G. Gouda, "Closure and convergence: A foundation of fault-tolerant computing," *IEEE Transactions on Software Engineering*, vol. 19, no. 11, pp. 1015–1027, 1993.
6. K. Apt and D. Kozen, "Limits for Automatic Verification of Finit-State Concurrent Systems," *Information Processing Letters*, vol. 22, no. 6, pp. 307–309, 1986.
7. I. Suzuki, "Proving properties of a ring of finite-state machines," *Information Processing Letters*, vol. 28, no. 4, pp. 213–214, 1988.
8. Y. Kwong, "Livelocks in parallel programs," *International Journal of Computer Mathematics*, vol. 10, no. 3-4, pp. 201–212, 1982.
9. M. Gouda and C. Chang, "Proving liveness for networks of communicating finite state machines," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 8, no. 1, pp. 154–180, 1986.
10. S. Leue and W. Wei, "Integer linear programming based property checking for asynchronous reactive system," *IEEE Transactions on Software Engineering*, no. 99, pp. 1–21, 2011.
11. J. Ouaknine, H. Palikareva, A. Roscoe, and J. Worrell, "Static livelock analysis in csp," *CONCUR 2011–Concurrency Theory*, pp. 389–403, 2011.
12. J. Blieberger, B. Burgstaller, and R. Mittermayr, "Static detection of livelocks in ada multitasking programs," in *Proceedings of the 12th international conference on Reliable software technologies*. Springer-Verlag, 2007, pp. 69–83.
13. E. Emerson and K. Namjoshi, "Reasoning about rings," in *CONFERENCE RECORD OF THE ACM SYMPOSIUM ON PRINCIPLES OF PROGRAMMING LANGUAGES*, vol. 22. ASSOCIATION FOR COMPUTING MACHINERY INC, 1995, pp. 85–94.
14. ——, "Automatic verification of parameterized synchronous systems," in *Computer Aided Verification*. Springer, 1996, pp. 87–98.
15. E. Emerson and V. Kahlon, "Reducing model checking of the many to the few," *Automated Deduction-CADE-17*, pp. 236–254, 2000.
16. ——, "Parameterized model checking of ring-based message passing systems," in *Computer Science Logic*. Springer, 2004, pp. 325–339.
17. M. G. Gouda and N. Multari, "Stabilizing communication protocols," *IEEE Transactions on Computers*, vol. 40, no. 4, pp. 448–458, 1991.
18. J. L. W. Kessels, "An exercise in proving self-stabilization with a variant function," *Information Processing Letters*, vol. 29, no. 1, pp. 39–42, 1988.
19. S. Shukla, D. Rosenkrantz, and S. Ravi, "Developing self-stabilizing coloring algorithms via systematic randomization," in *Proceedings of the International Workshop on Parallel Processing*. Citeseer, 1994, pp. 668–673.
20. A. Farahat and A. Ebnenasir, "Local reasoning for global convergence of parameterized rings," Michigan Technological University, Tech. Rep. CS-TR-11-04, November 2011, http://www.cs.mtu.edu/html/tr/11/11-04.pdf.
21. ——, "Local reasoning for global convergence of parameterized rings," in *Proceedings of the 32nd International Conference on Distributed Computing Systems*.
22. M. Herlihy and N. Shavit, "The topological structure of asynchronous computability," *Journal of the ACM (JACM)*, vol. 46, no. 6, pp. 858–923, 1999.

23. E. W. Dijkstra, *A Discipline of Programming.* Prentice-Hall, 1990.
24. P. Godefroid, "Using partial orders to improve automatic verification methods," in *Computer-Aided Verification.* Springer, 1991, pp. 176–185.
25. S. Leue, A. Ştefănescu, and W. Wei, "A livelock freedom analysis for infinite state asynchronous reactive systems," *CONCUR 2006–Concurrency Theory*, pp. 79–94, 2006.
26. ——, "Dependency analysis for control flow cycles in reactive communicating processes," *Model Checking Software*, pp. 176–195, 2008.
27. A. Ebnenasir, "Automatic synthesis of fault-tolerance," Ph.D. dissertation, Michigan State University, May 2005.